Queue

So far, we have learnt the data structures list, stack, circular list. We have observed that these are all abstract data types supporting a well-defined interface. We have also observed that each of these abstract data types could have different implementations. These data structures are found to be useful in different applications based on their interfaces.

Consider the case of printing a few files on a printer. We can fire the print command on many files all at once. How would the printer handle this? Would the files get mixed up? Not at all. The files are printed in the order in which they arrive at the printer. Whichever comes first will be printed first. The printer follows "First Come First Served" principle. To support this behaviour, the files are put in a **queue**. The files are serviced from the **front** and are attached at the **rear**.

Queue is a data structure which has two ends – normally called the rear and the front. The components are added at the rear and removed from the front. We say that the queue exhibits the behaviour - FIFO – First In First Out.

Interface of the queue:

Apart from initialization and deinitialization, the queue supports the following operations.

initialize()	Create an empty queue
enqueue(x)	Insert x at the rear
dequeue()	Remove the element at the front
front() / peek()	Examine the front element
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations
destroy() / deinitialize()	Release resources

Implementation : We can implement the queue in different ways. We will discuss “implementing the queue using a list” in this section. We will discuss “implementing the queue using an array in the next section”.

The queue shall have the following structure. It has two pointers referred to as front_ and rear_.

```
struct queue
{
   node_t* front_;
   node_t* rear_;
};
typedef struct queue queue_t;
```

The node shall have the component and a pointer to a node – it is a classical self-referential structure.

```
struct node
{
   int key_;
   struct node* next_;
};
typedef struct node node_t;
```

For illustration purpose, the component type is chosen as an int.

Design consideration : How to link the nodes?

The two major operations on a queue are :

Adding at the rear : enqueue

Removing from the front : dequeue.

How to support these operations efficiently?

Design 1 :

One possible arrangement could be as follows.

Rear of the queue will point to the last node. The last node will point to the node which came before that. This in turn points to the node which came before that and so on. The front will point to the first node.

You will observe that enqueue will require a few operations. Create a node. Populate it. Make the pointer field take the value of rear of the queue. Change rear to point to the new node created.

So far so fine.

How about removing the first node from the front? This would require that the front should point to the node after the first node. But the pointer field in that node is grounded. As this is a singly linked data structure, we cannot move back. The only way we can satisfy this is by travelling all the way from the rear to the front. So, deque would take as many operations as the number of nodes in the queue. This is not acceptable.

Design 2 :

Let the node pointed to by the front of the queue point to the next node. Each node shall point to the node which came later. The rear of the queue shall point to the last node whose pointer field is grounded.

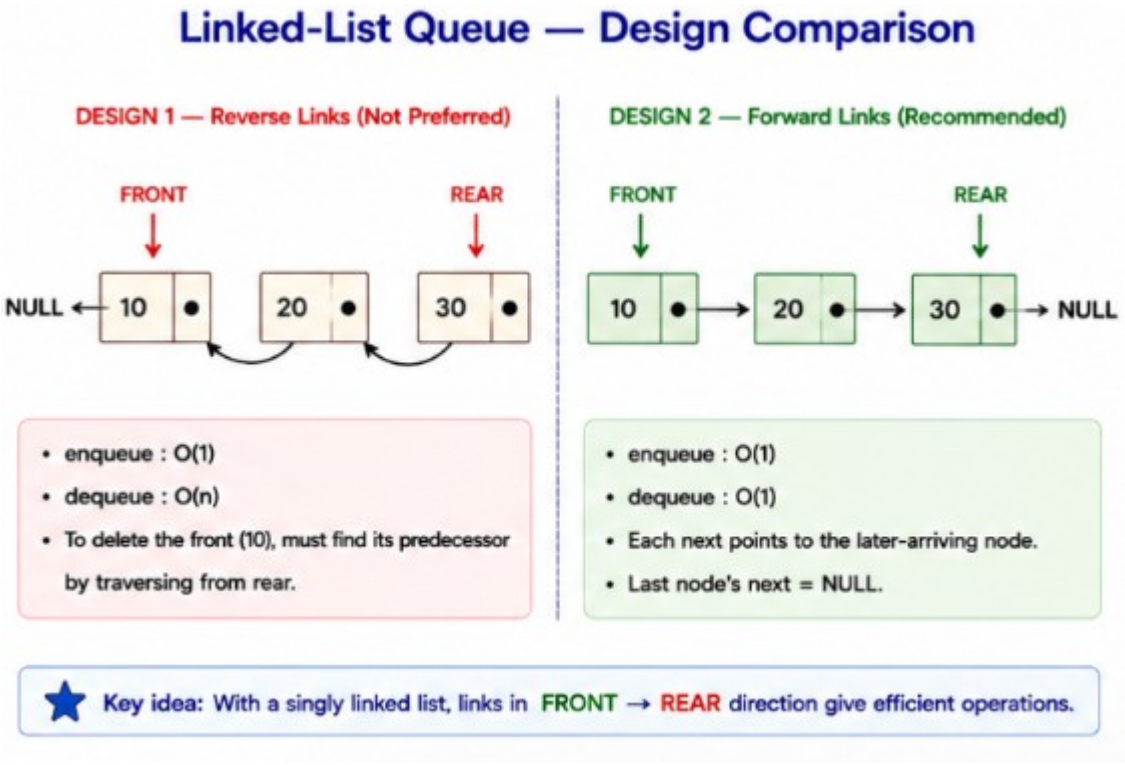
Enqueue requires that we create a node, populate it, change the pointer field of the node pointed to by rear to this new node, ground the pointer field in the new node, change rear to point to this new node.

This requires just a few operations. So fine.

Dequeue requires that we change the front to take the pointer in the first node so that it points to the next node.

This also requires just a few operations. So fine.

So, in out design each node will point to the node which came next.



Implementation:

- **Initialize the list**

  Both the front and the rear of the queue are initialized to NULL.

  ```
  void init_queue(queue_t *ptr_queue)
  {
      ptr_queue->rear_ = ptr_queue->front_ = NULL;
  }
  ```

- **Deinitalize the list**

  This requires a traversal of the routine and freeing of each node. We could start from the front. Make that the pres node. Store the next of pres in a pointer. Delete pres. Make pres take the stored pointer value. Repeat this until all nodes are deleted.

  Could we call deque until the queue becomes empty?

  ```
  void deinit_queue(queue_t *ptr_queue)
  {
      while(! is_empty(ptr_queue))
      {
              deque(ptr_queue);
      }
  ```

```
    ptr_queue->rear_ = ptr_queue->front_ = NULL;
}
```

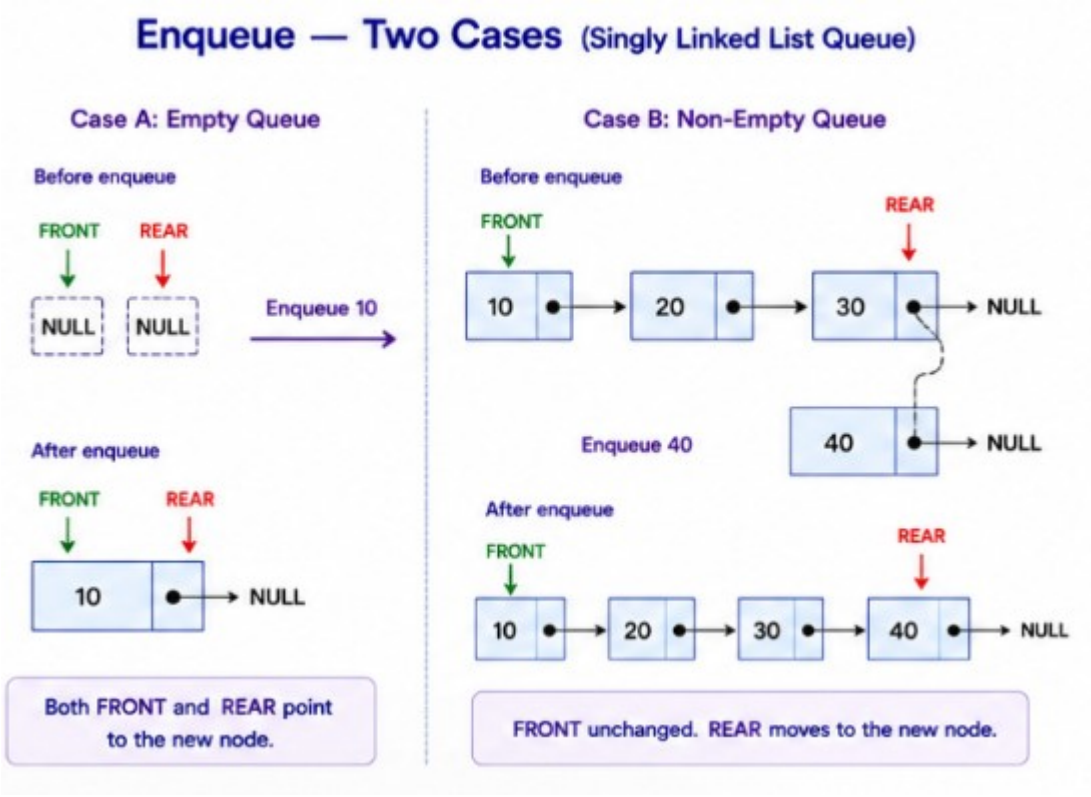
- **Enqueue**

  If the malloc operation fails, it means that the queue is full. It is an unusual case – an exception. We do not know what the user wants to do. So, we will give a message to the user.

  We shall consider two cases .

  1. empty queue
     - create a node with the component.
     - Make rear of the queue point to it.
     - Ground the pointer in the new node created.
     - **Make front also point to the new node created**
       When there is only one node, it is both the front and the rear!

  2. queue with some nodes
     - create a node with the component.
     - Make the pointer in the node to which rear points to, to point to the new node created.
     - Make rear of the queue point to the new node.



```
void enque(queue_t *ptr_queue, int key)
{
    if(! is_full(ptr_queue))
    {
        node_t* temp = create_node(key);
        temp->next_ = NULL;
        if(ptr_queue->rear_ == NULL)
        {
            ptr_queue->front_ = temp;
        }
        else
        {
            ptr_queue->rear_->next_ = temp;
        }
        ptr_queue->rear_ = temp;
    }
    else
    {
        printf("queue full; cannot push\n");
    }
}
```

Create a node :

Enque calls this function to create a node. It is a good idea to develop a function for a particular purpose. This function is also not required for the client. By making the function, we could make this available in the file and not outside this file. Such a function is prefixed with the key static – which in this context means that the function is visible in this file alone – technically said to have internal linkage.

```
static node_t* create_node(int key)

{

        node_t* temp = (node_t*)malloc(sizeof(node_t));

        temp->key_ = key;

        return temp;

}
```

- **Dequeue** :

Which cases should we consider.

1. Empty queue : We cannot remove an element from an empty queue. We will give a message to the user.
2. Queue with some nodes.
   Please watch these steps. The order matters.
   - Copy the front of the queue to a local pointer variable say temp.
   - Copy the component in this node to a local variable say result
   - Change front to point to the next node by copying the pointer value in the first node
   - Remove the node to which temp points to.
   - Return the component stored in the variable result.

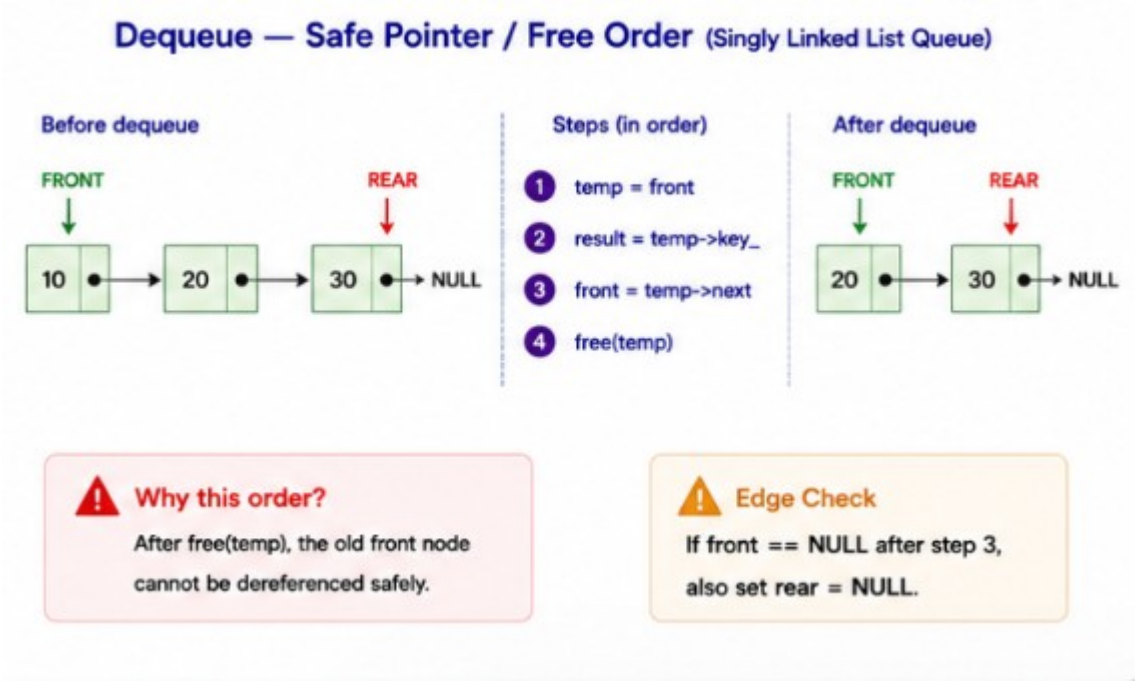
   It is interesting to think as to what happens if the order is modified!

   Why not free the node to which front points to?

   Why not just change the front?

   This procedure has a serious bug. Think of what happens if you are the only person in a queue. So you are the front and the rear. What if you are served? The front shall no more point to you. It is grounded. What about the rear?

   When there is only one node in the queue and it is dequeued, then the rear also will change!



```
int deque(queue_t *ptr_queue)
{
	if(! is_empty(ptr_queue))
	{
		int key;  node_t* temp = ptr_queue->front_;
		key = temp->key_;

		ptr_queue->front_ = temp->next_;
		if(ptr_queue->front_ == NULL)
		{
			ptr_queue->rear_ = NULL;
		}
		free(temp);
		return key;
	}
	else
	{
		printf("queue empty; cannot pop\n");
		return '\0';
	}
}
```

Dequeue is returning 0 when it fails. It is possible in our case as the component is integer. We could avoid returning any value from deque.

- **is_empty :**

  The queue is empty if the front is grounded. Can we say the same if rear is grounded?

  ```
  int is_empty(queue_t *ptr_queue)

  {

      return ptr_queue->front_ == NULL;

  }
  ```

- **Is_full**:
  This is tricky. Unless malloc fails, we will not know whether queue is full. If we call malloc and it fails, it is a bit too late to signal that the queue is full. We try to allocate. If it succeeds, we will conclude that the queue is not full. We free the memory allocated as it is not required now and is allocated just to check whether the allocation succeeds.

  ```
  int is_full(queue_t *ptr_queue)
  {
     node_t* temp;
       temp = (node_t*)malloc(sizeof(node_t));
     if (temp == NULL) {
        return 1;
     }
     free(temp);
     return 0;
  }
  ```

  The client could use a menu driven program to experiment with enqueue and dequeue in various permutations.

```
printf("opt : 1 : enque; 2 : deque ; 0 : quit\n");
scanf("%d", &opt);
while(opt)
{
        switch(opt)
        {
                case 1 : scanf("%d", &val);
                                enque(&q, val);
                                break;
                case 2 : val = deque(&q);
                                if(val != 0)
                                        printf("deque %d\n", val);
                                break;
        }
        printf("opt : 1 : enque; 2 : deque ; 0 : quit\n");
        scanf("%d", &opt);
}
```

Observe that there is an input before the loop and input at the end of the loop. This is a characteristic of any menu driven program. We could avoid the duplication of the code by delegating the work to a function.

Applications of queue :

- CPU scheduling : processes in batch processing usually wait in a queue
- Printer Spooler : collects the jobs for printing in a queue
- Breadth First search : Looking for a key in a tree structure
- Simulation models : queues to simulate random events in queueing theory
- Message queues : sharing information in operating systems
- Toll booth : waiting of vehicles to pass through a toll booth